CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY
An Autonomous Institute | Affiliated to Osmania University
Kokapet Village, Gandipet Mandal, Hyderabad, Telangana-500075, www.cbit.ac.in

COMMITTED TO RESEARCH, INNOVATION AND EDUCATION 48 years

FULL STACK WEB DEVELOPMENT
(22CAC09N)

CASE STUDY - 2 REPORT

CineWorld — Movie Ticket Booking & Resale Platform

SUBMITTED BY

Name : A.Bheeshma Shankar & Ch.Pranav
Roll Number : 160124748093 & 160124748099

B.E. CSM-2
V SEMESTER
III YEAR

ACADEMIC YEAR: 2026-2027

SUBMITTED TO

Ms. G. Baby Lakshmi Prasanna

Date of Submission: 27/10/2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
(ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING)

CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY

(A) GANDIPET, HYDERABAD - 500075

TABLE OF CONTENTS

1. INTRODUCTION 3
   1.1 Background 3
   1.2 Problem Statement 3
   1.3 Importance in Full Stack Development 3
   1.4 Real-World Applications 4
   1.5 Objectives of the Case Study 4
2. TECHNOLOGIES AND METHODS 4
   2.1 Node.js 4
   2.2 Express.js 5
   2.3 MongoDB and Mongoose 5
   2.4 REST API Architecture 5
   2.5 Authentication and Middleware 5
   2.6 Razorpay Test Mode 6
   2.7 MongoDB Compass and API Testing 6
   2.8 Summary of Technologies 6
3. BACKEND ARCHITECTURE 7
   3.1 Overall Backend Architecture 7
   3.2 Request–Response Cycle 7
   3.3 Database Architecture 8
   3.4 Authentication Flow 8
   3.5 Booking and Seat Reservation Flow 9
   3.6 Payment Verification Flow 10
   3.7 Resale Workflow 11
4. IMPLEMENTATION 12
   4.1 Project Workspace Structure 12
   4.2 Express Server Configuration 13
   4.3 Middleware Configuration 13
   4.4 Mongoose Models and Schema Design 14
   4.5 REST API Endpoints 14
   4.6 CRUD Operations 15
   4.7 Authentication and Authorization 16
   4.8 Booking and Seat Management 16

4.9 Razorpay Test Payment Integration 17
4.10 Ticket Resale Management 18
4.11 Environment Configuration 18
4.12 MongoDB Database Initialization 19
5. RESULTS AND EXECUTION OUTPUTS 19
5.1 Backend Server Startup 19
5.2 Health-Check API 19
5.3 Movie Retrieval Using GET 20
5.4 Showtime Retrieval Using GET 20
5.5 MongoDB Database and Collections 21
5.6 User Registration and Login 22
5.7 Account Login Using POST 22
5.8 Movie Booking and Seat Selection 23
5.9 Booking Confirmation and MongoDB Records 23
5.10 Razorpay Test-Payment Integration 24
5.11 Movie Update Using PUT 25
5.12 Movie Deletion Using DELETE 25
5.13 Ticket Resale System 26
5.14 Final VS Code Project Structure 27
5.15 Result Summary 27
6. CONCLUSION 28
6.1 Summary of Findings 28
6.2 Technical Findings 28
6.3 Learning Outcomes 28
6.4 Current Implementation Limitations 28
6.5 Future Scope 29
6.6 Final Conclusion 29

Case Study 2: CineWorld – Movie Ticket Booking and Resale System

1. INTRODUCTION

1.1 Background

Online movie ticketing is one of the most common transaction-based web applications. A user browses movies, picks a show, chooses seats, pays, and receives a ticket. Behind this simple flow, the server has to manage user accounts, show schedules, seat availability, payment confirmation and booking records.

CineWorld is a full stack movie ticket booking and resale system built as a case study. The backend is written in Node.js with Express.js and stores its data in a MongoDB database named `cineworld` through Mongoose. Users can register, log in, book seats for a show, pay through Razorpay in test mode, and list a confirmed ticket seat for resale. Static HTML, CSS and JavaScript pages in the `public` folder are served by the same Express server and call the backend APIs. This report focuses mainly on the backend.

1.2 Problem Statement

A ticket booking system must handle several problems correctly:

- Two users must never be able to reserve the same seat for the same show.
- Only logged-in users should be able to book, pay, or resell tickets.
- A booking should be confirmed only after the payment is verified on the server, not just because the browser says it was paid.
- Users who cannot attend should be able to pass on a ticket in a controlled way.
- Management actions on movies, such as adding or deleting them, should be limited to authorised users.

The case study addresses these problems with a REST API backend that uses database-level uniqueness rules, token-based authentication, signature-based payment verification and a controlled resale workflow.

1.3 Importance in Full Stack Development

This project covers the main layers of a full stack application in one place:

- **Server layer:** Node.js runtime, Express routing, middleware and error handling.
- **Data layer:** MongoDB collections, Mongoose schemas, validation and unique indexes.
- **Security layer:** password hashing, JSON Web Tokens (JWT) stored in HttpOnly cookies, role checks, and HMAC payment-signature verification.
- **Integration layer:** a third-party payment gateway (Razorpay test mode).
- **Client layer:** static pages that consume the REST APIs using `fetch`.

It also shows practical backend concerns such as handling race conditions on seat booking and keeping a clear difference between implemented and demo-only behaviour.

1.4 Real-World Applications

The same design ideas apply to many real systems:

Table 1.1: Real-world applications of the concepts used in CineWorld

Domain	Similar use of the same concepts
Cinema and event ticketing	Seat selection, show schedules, booking confirmation
Transport booking	Bus, train and flight seat reservation
Online payments	Gateway order creation and signature verification
Ticket marketplaces	Controlled resale and ownership transfer
Appointment and slot booking	Preventing double booking with unique indexes

1.5 Objectives of the Case Study

1. Build a Node.js and Express.js backend that runs on port 5000 and exposes a health-check API.
2. Design Mongoose models for User, Movie, Show, Booking, Payment, SeatReservation and Resale in a MongoDB database named `cineworld`.
3. Implement user registration, login and authentication-protected routes.
4. Implement movie routes with GET, POST, PUT and DELETE operations, with write operations limited to admin users.
5. Implement booking creation and retrieval with seat validation and booking status management.
6. Prevent duplicate seat reservations for a show using a unique compound index.
7. Integrate Razorpay in test mode for order creation and payment-signature verification.
8. Implement resale listing, retrieval, withdrawal and a demo ticket ownership transfer.
9. Inspect the stored data using MongoDB Compass and test the APIs using PowerShell.

2. TECHNOLOGIES AND METHODS

2.1 Node.js

Node.js is a JavaScript runtime built on Chrome's V8 engine. It runs JavaScript outside the browser and uses an event-driven, non-blocking input/output model, which suits API servers that spend most of their time waiting on the database or network.

In CineWorld, Node.js runs `server.js`, loads environment variables with `dotenv`, and uses built-in modules: `path` for file locations, `fs` for file appending, `events` for the `EventEmitter` class, and `crypto` for random ticket codes and HMAC signature checks. Dependencies are managed with npm and listed in `package.json`.

2.2 Express.js

Express.js is a minimal web framework for Node.js. It provides routing, middleware chaining and response helpers. CineWorld uses Express to:

- Mount five route groups under `/api/auth`, `/api/movies`, `/api/bookings`, `/api/payments` and `/api/resale`.
- Serve the static frontend from the `public` folder.
- Parse JSON and URL-encoded request bodies and cookies.
- Return a JSON 404 response for unknown routes and a central JSON error response for server errors.

2.3 MongoDB and Mongoose

MongoDB is a document-oriented NoSQL database that stores data as JSON-like documents. Mongoose is an Object Data Modelling (ODM) library that adds schemas, validation, references between collections and query helpers.

CineWorld uses Mongoose for the following:

- Schema definitions with `required`, `enum`, `min`, `unique`, `match` and default values.
- References using `ObjectId` and `ref`, with `populate()` to join related data.
- Automatic `createdAt` and `updatedAt` fields through `timestamps: true`.
- Compound unique indexes on Show and SeatReservation.

2.4 REST API Architecture

REST (Representational State Transfer) organises an API around resources and HTTP methods. CineWorld follows this style: `GET` retrieves data, `POST` creates data or triggers an action, `PUT` updates a movie, and `DELETE` removes a movie. Responses are JSON, and standard status codes are used in the code, for example 201 for created, 400 for bad input, 401 for unauthenticated, 403 for forbidden, 404 for not found and 409 for conflicts.

2.5 Authentication and Middleware

Middleware functions in Express run between the request and the final handler. CineWorld uses them for body parsing, cookie parsing, static files, authentication and role checking.

Authentication works as follows:

- Passwords are hashed with **bcryptjs** (cost factor 12) and only the hash is stored.
- On successful login or registration, the server signs a **JWT** that holds the user id and role and expires in 7 days.
- The JWT is sent as an **HttpOnly cookie** named `cineworld_token` with `sameSite: lax`. The `secure` flag is switched on when `NODE_ENV` is `production`.
- The `requireAuth` middleware accepts the token from the cookie or from an `Authorization: Bearer` header, verifies it, and attaches the decoded user to `req.user`.

- The `requireAdmin` middleware allows access only when the role is `admin`.

2.6 Razorpay Test Mode

Razorpay is a payment gateway. Test mode lets developers run the full payment flow with test keys and no real money. In CineWorld the backend:

1. Creates a Razorpay order for the booking amount (converted to paise).
2. Returns the order details to the client for checkout.
3. Receives the payment id, order id and signature from the client.
4. Recomputes the signature on the server using HMAC-SHA256 and compares it using `crypto.timingSafeEqual`.

This is a **test-mode integration only**. It is not configured as a production payment system.

2.7 MongoDB Compass and API Testing

- **MongoDB Compass** is the graphical client used to inspect the `cineworld` database, its collections, documents and indexes.
- **PowerShell** was used to start the backend and to send test requests to the APIs.

2.8 Summary of Technologies

Table 2.1: Summary of technologies used

Technology	Role in CineWorld
Node.js	Runtime for the backend server
Express.js	Routing, middleware, static files, error handling
MongoDB	Database named cineworld
Mongoose	Schemas, validation, references, unique indexes
bcryptjs	Password hashing
jsonwebtoken	Signing and verifying JWT tokens
cookie-parser	Reading the authentication cookie
dotenv	Loading configuration from .env
razorpay (SDK)	Creating test-mode payment orders
Node crypto	Ticket codes and signature verification
Node events and fs	Custom server-started event written to a log file
nodemon (dev)	Auto-restart during development
MongoDB Compass	Database inspection
PowerShell	Starting the server and testing APIs

3. BACKEND ARCHITECTURE

3.1 Overall Backend Architecture

The backend follows a layered structure: the entry file configures the app, route files hold the endpoint logic, middleware handles authentication, and models define the data. Only the payments route calls Razorpay, and every route group reads or writes the `cineworld` database.

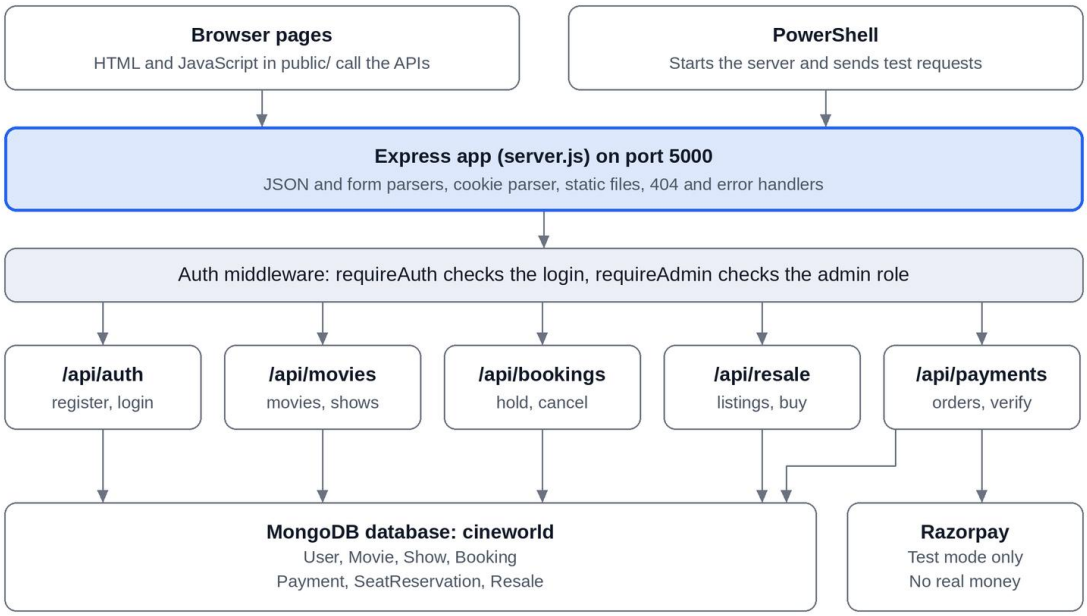


Figure 3.1: Overall backend architecture of CineWorld.

3.2 Request–Response Cycle

1. The client sends an HTTP request to port 5000.
2. Express runs the global middleware: raw body parser for the webhook path, JSON parser (100 KB limit), URL-encoded parser, cookie parser and static file handler.
3. The router matches the path, for example `/api/bookings`.
4. Protected routes run `requireAuth` first. If the token is missing or invalid, the server responds with 401.
5. The route handler validates input, runs Mongoose queries and builds the response.
6. Errors are passed with `next(error)` to the central error handler, which returns JSON. Unknown routes return `404 Route not found`.

3.3 Database Architecture

The database is `cineworld`. It holds seven collections, related as shown in Figure 3.2 and described in Table 3.1.

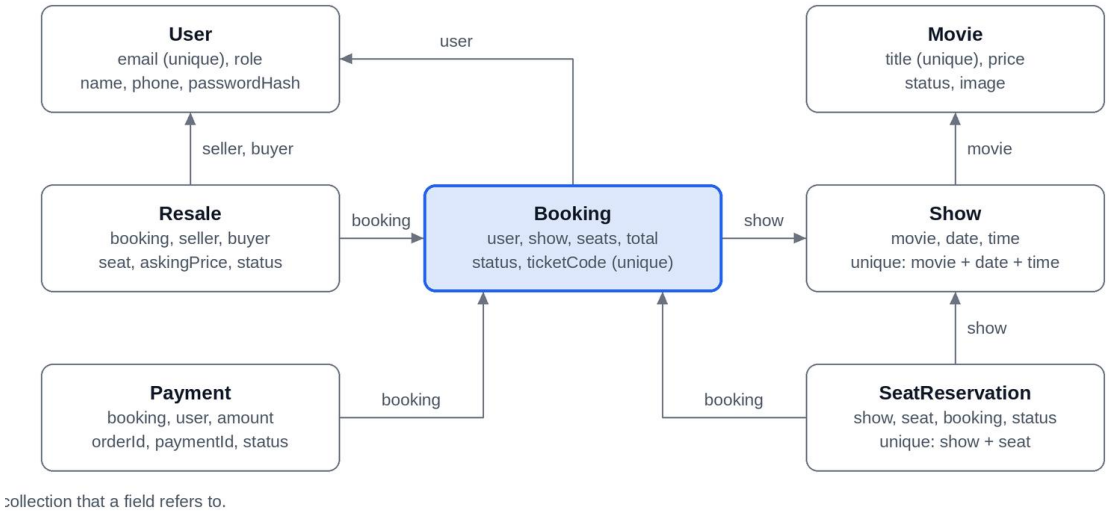


Figure 3.2: Relationships between the seven collections of the cineworld database.

Table 3.1: Collections of the cineworld database

Collection	Purpose	Key fields	Relationships and indexes
User	Registered accounts	name, email, phone, passwordHash, role (customer or admin)	email is unique
Movie	Movies available	title, price, image, status (now-showing or coming-soon), description	title is unique
Show	A movie screening	movie, date, time, screen, capacity (default 40)	References Movie; unique on movie + date + time
Booking	A user's ticket booking	user, show, movieTitle, showDate, showTime, seats, total, status, paymentId, ticketCode	References User, Show, Payment; ticketCode unique and sparse
Payment	Payment attempt record	booking, user, orderId, paymentId, signature, amount, currency, status (CREATED, PAID, FAILED)	References Booking and User
SeatReservation	Seat claimed for a show	show, seat, booking, status (HELD or BOOKED)	Unique on show + seat
Resale	A seat listed for resale	booking, seller, buyer, seat, movieTitle, showDate, showTime, askingPrice, status (LISTED, SOLD, WITHDRAWN)	References Booking and User

Booking status values are `PENDING_PAYMENT`, `CONFIRMED` and `CANCELLED`.

3.4 Authentication Flow

1. The user sends name, email, optional 10-digit phone and password to `POST /api/auth/register`.

2. The server validates the input, checks for an existing email, hashes the password and creates the user.
3. A JWT is signed and set as an HttpOnly cookie. The response contains a safe user object without the password hash.
4. On `POST /api/auth/login`, the server finds the user by email and compares the password with the stored hash.
5. For protected routes, `requireAuth` reads and verifies the token. `GET /api/auth/me` returns the current user.
6. `POST /api/auth/logout` clears the cookie.

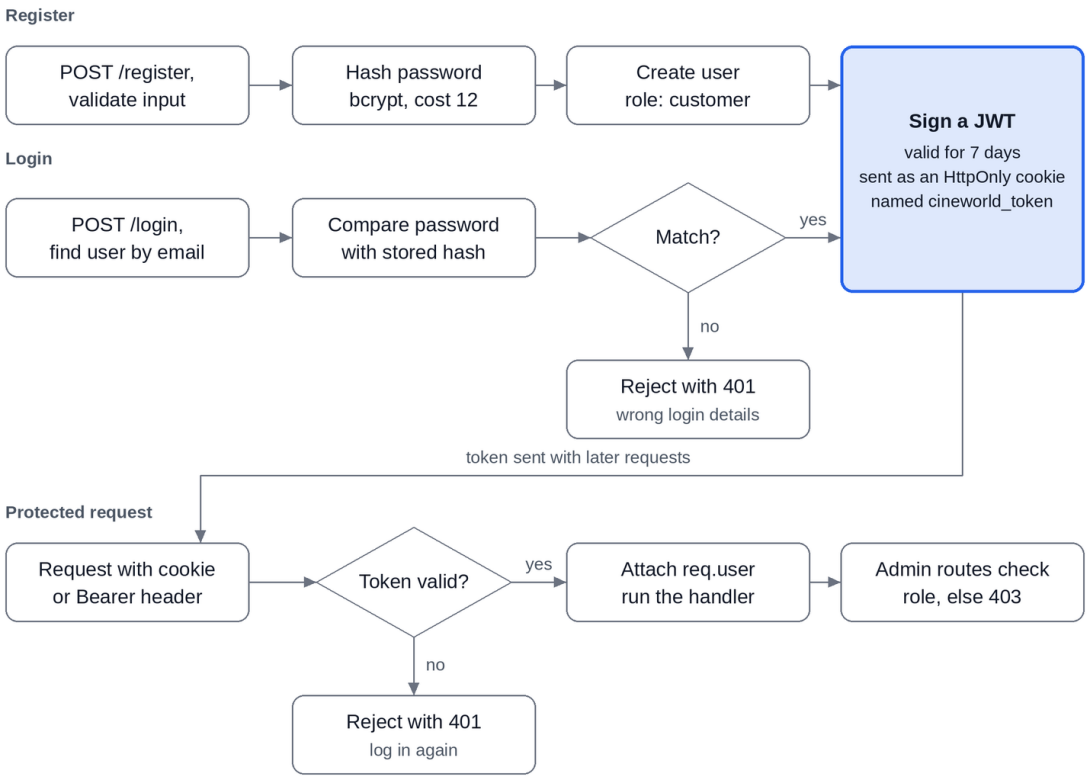


Figure 3.3: Authentication flow from registration and login to a protected request.

3.5 Booking and Seat Reservation Flow

1. The user sends `movieId`, `date`, `time` and `seats` to `POST /api/bookings`.
2. The server checks that all fields exist, that seat codes are unique and match the pattern rows A–E and numbers 1–8 (for example `A1`, `E8`), that the movie is `now-showing`, and that a matching show exists.
3. A Booking is created with status `PENDING_PAYMENT`, a total (price × number of seats) and a ticket code beginning with `CW-`.
4. The server inserts one SeatReservation per seat with status `HELD`.
5. If the unique index on show + seat rejects any seat, the server removes the partial reservations and the booking, and returns 409 with a message that a seat was just taken.

6. After payment verification, the reservations change to `BOOKED` and the booking becomes `CONFIRMED`.
7. A pending booking can be cancelled with `POST /api/bookings/:id/cancel`, which sets it to `CANCELLED` and deletes the held seats.

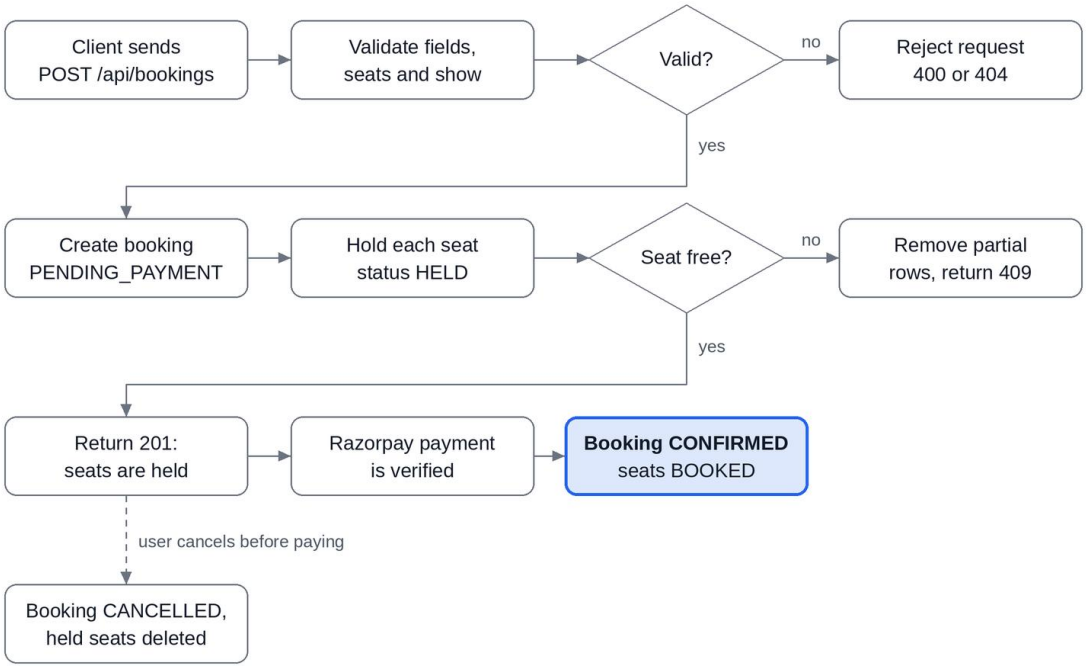


Figure 3.4: Booking and seat reservation flow, from request to confirmed or cancelled booking.

3.6 Payment Verification Flow

1. The client calls `POST /api/payments/create-order` with a booking id.
2. The server confirms the booking belongs to the user and is pending, then creates a Razorpay order (amount in paise, currency INR) and stores a Payment record with status `CREATED`.
3. The server returns the order details and the public key id.
4. After checkout, the client sends the order id, payment id, signature and booking id to `POST /api/payments/verify`.
5. The server computes HMAC-SHA256 over `order_id|payment_id` using the secret key and compares it with the received signature.
6. If it matches, the Payment becomes `PAID`, the Booking becomes `CONFIRMED` and the seats become `BOOKED`. If not, the Payment becomes `FAILED` and the server returns 400.

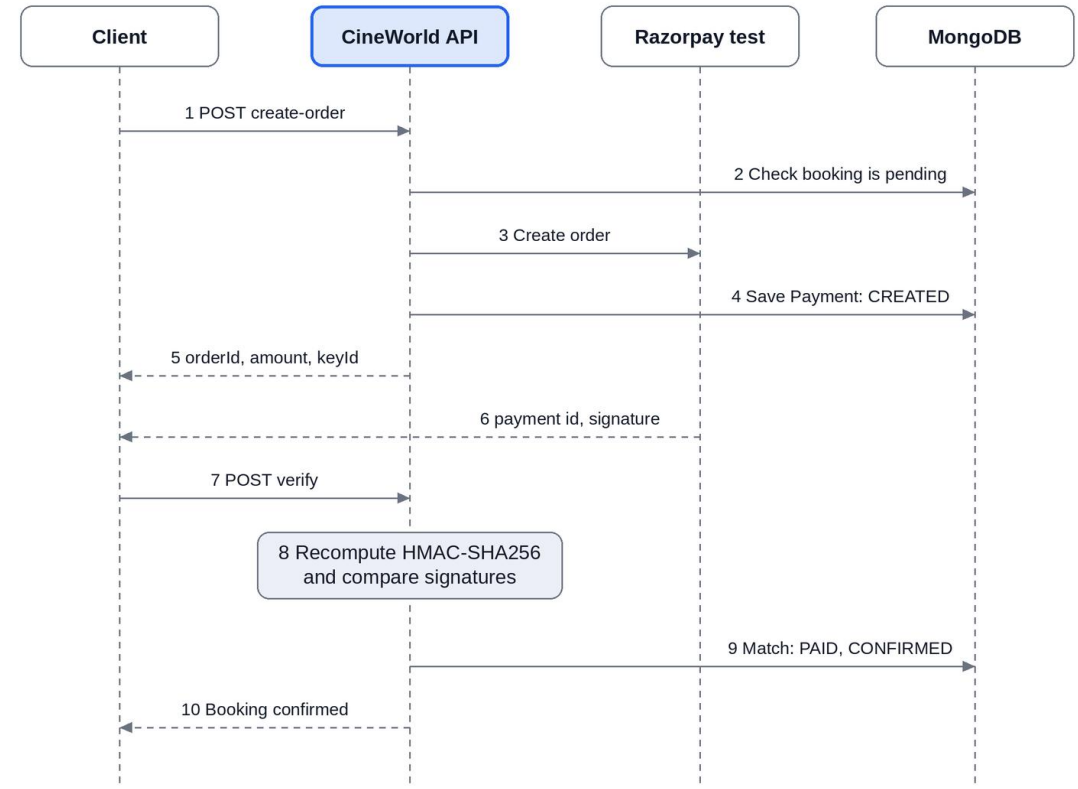


Figure 3.5: Razorpay test-mode order creation and payment verification sequence.

3.7 Resale Workflow

1. The owner lists one seat from a **confirmed** booking with an asking price using `POST /api/resale`.
2. The server checks ownership, seat validity, that the seat belongs to the booking, that the seat is still `BOOKED`, and that it is not already listed.
3. Other users view active listings through `GET /api/resale`.
4. A buyer calls `POST /api/resale/:id/buy`. The server first claims the listing atomically by changing its status from `LISTED` to `SOLD`, so two buyers cannot take the same listing. A seller cannot buy their own listing.
5. The seat is removed from the seller's booking, a new confirmed booking (ticket code starting `CW-R-`) is created for the buyer, and the seat reservation is moved to the new booking.
6. If a step fails, the code attempts a rollback of the changes.
7. The seller can withdraw an active listing with `POST /api/resale/:id/withdraw`.

> **Important:** The resale purchase is a **demo ownership transfer**. No resale payment is collected. The response itself reports `paymentCollected: false`.

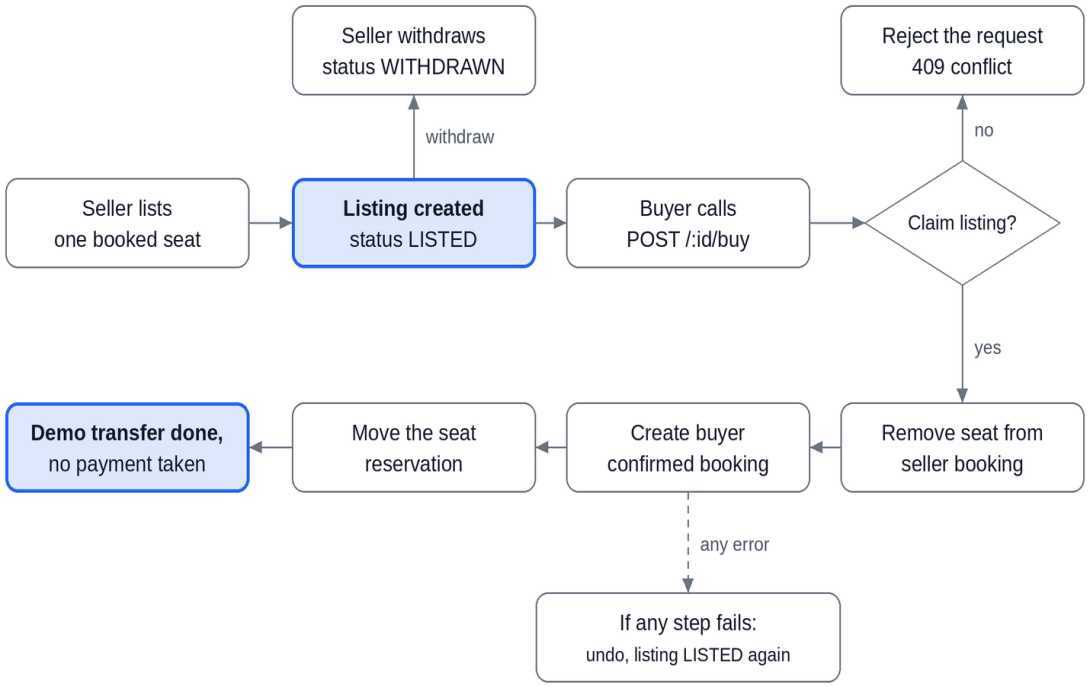


Figure 3.6: Resale listing, withdrawal and demo ownership transfer workflow.

4. IMPLEMENTATION

4.1 Project Workspace Structure

Table 4.1: Project workspace structure

Path	Description
server.js	Application entry point, middleware, route mounting, database connection, startup event
middleware/auth.js	requireAuth and requireAdmin
models/	User.js, Movie.js, Show.js, Booking.js, Payment.js, SeatReservation.js, Resale.js
routes/	auth.js, movies.js, bookings.js, payments.js, resale.js
scripts/seed.js	Loads movies and 14 days of shows
public/	Static HTML, CSS and JavaScript pages and movie images
.env.example	Template for configuration (no secrets)
package.json	Dependencies and npm scripts (start, dev, seed)
server-events.log	Log file appended on server start
README.md	Setup and API summary

4.2 Express Server Configuration

The server reads the port from the environment and defaults to 5000. Startup is wrapped in an async function: it checks that `MONGODB_URI` and `JWT_SECRET` are set, connects to MongoDB, prints "MongoDB connected", and only then starts listening. If startup fails, it prints the error and exits with code 1.

A custom `EventEmitter` event named `server-started` is emitted once the server is listening. Its handler appends a timestamped line to `server-events.log` using the asynchronous `fs.appendFile`. This is implemented in `server.js`.

Code snippet: startup sequence, custom event and log file in server.js.

```
const EventEmitter = require('events');
const fs = require('fs');
const serverEvents = new EventEmitter();

serverEvents.on('server-started', (port) => {
  const line = `${new Date().toISOString()} started on ${port}\n`;
  fs.appendFile('server-events.log', line, (err) => {
    if (err) console.error('Log write failed:', err.message);
  });
});

async function start() {
  try {
    if (!process.env.MONGODB_URI || !process.env.JWT_SECRET) {
      throw new Error('MONGODB_URI and JWT_SECRET must be set');
    }
    await mongoose.connect(process.env.MONGODB_URI);
    console.log('MongoDB connected');
    app.listen(PORT, () => {
      console.log(`Server running on port ${PORT}`);
      serverEvents.emit('server-started', PORT);
    });
  } catch (error) {
    console.error('Startup failed:', error.message);
    process.exit(1);
  }
}
start();
```

4.3 Middleware Configuration

Table 4.2: Middleware configuration order

Order	Middleware	Purpose
1	express.raw on /api/payments/webhook	Keeps the raw body for webhook signature checking
2	express.json (100 KB limit)	Parses JSON bodies
3	express.urlencoded	Parses form bodies
4	cookie-parser	Reads the authentication cookie
5	express.static('public')	Serves the frontend pages

Order	Middleware	Purpose
6	Route modules	API endpoints
7	404 handler	JSON “Route not found”
8	Error handler	Central JSON error response; hides internal messages for 500 errors

4.4 Mongoose Models and Schema Design

All seven models are in the `models` folder with required-field validation and timestamps. Notable design choices:

- **User:** name length 2–80, unique lowercase email, phone must be exactly 10 digits, role defaults to `customer`.
- **Movie:** unique title, price at least 1, status limited to two values.
- **Show:** unique combination of movie, date and time prevents duplicate shows.
- **SeatReservation:** unique combination of show and seat is the main guard against double booking.
- **Booking:** stores a snapshot of movie title, date and time with the seats, total and a unique ticket code.
- **Payment:** stores order id, payment id, signature, amount and status.
- **Resale:** stores seller, optional buyer, seat, asking price and status.

The collection summary is given in Table 3.1.

Code snippet: SeatReservation model with the unique compound index (models/SeatReservation.js).

```
const { ObjectId } = mongoose.Schema.Types;

const seatReservationSchema = new mongoose.Schema({
  show:    { type: ObjectId, ref: 'Show', required: true },
  seat:    { type: String, required: true, uppercase: true },
  booking: { type: ObjectId, ref: 'Booking', required: true },
  status:  { type: String, enum: ['HELD', 'BOOKED'], default: 'HELD' }
}, { timestamps: true });

// One reservation per seat per show
seatReservationSchema.index({ show: 1, seat: 1 }, { unique: true });

module.exports = mongoose.model(
  'SeatReservation', seatReservationSchema);
```

4.5 REST API Endpoints

Table 4.3 lists the endpoints present in the route files. The last column points to the section that holds the execution evidence.

Table 4.3: REST API endpoints of CineWorld

Method	Endpoint	Access	Purpose	Evidence
GET	/api/health	Public	Health check	Section 5.2
POST	/api/auth/register	Public	Create account	Section 5.6
POST	/api/auth/login	Public	Log in	Section 5.6
GET	/api/auth/me	Logged in	Current user	Section 5.6
GET	/api/movies	Public	List movies (optional status filter)	Section 5.3
GET	/api/movies/shows	Public	List shows (optional movieId, date)	Section 5.4
GET	/api/movies/shows/:id/seats	Public	Booked seats for a show	Section 5.4
PUT	/api/movies/:id	Admin	Update a movie	Section 5.11
DELETE	/api/movies/:id	Admin	Delete a movie	Section 5.12
POST	/api/bookings	Logged in	Create booking and hold seats	Section 5.7
GET	/api/bookings/mine	Logged in	List own bookings	Section 5.7
GET	/api/bookings/:id	Logged in	Get one own booking	Section 5.7
POST	/api/payments/create-order	Logged in	Create Razorpay test order	Section 5.8
POST	/api/payments/verify	Logged in	Verify payment signature	Section 5.8
GET	/api/resale	Public	Active resale listings	Section 5.9
GET	/api/resale/mine	Logged in	Own listings	Section 5.9
POST	/api/resale	Logged in	Create listing	Section 5.9
POST	/api/resale/:id/buy	Logged in	Demo ownership transfer	Section 5.9
POST	/api/resale/:id/withdraw	Logged in	Withdraw listing	Section 5.9

The webhook endpoint exists in the code, but it only responds when `RAZORPAY_WEBHOOK_SECRET` is configured and, as written, it only validates the signature and replies `received: true`. It does not update any booking.

4.6 CRUD Operations

Table 4.4: CRUD coverage by resource

Resource	Create	Read	Update	Delete
User	POST /api/auth/register	GET /api/auth/me	Not implemented	Not implemented
Movie	POST /api/movies (admin)	GET /api/movies	PUT /api/movies/:id (admin)	DELETE /api/movies/:id (admin)
Show	Created by scripts/seed.js	GET /api/movies/shows	Not implemented	Not implemented
Booking	POST /api/bookings	GET /api/bookings/mine, /:id	Status changes via cancel and payment verify	Not exposed (cancel sets status)
Resale	POST /api/resale	GET /api/resale, /mine	Buy and withdraw change status	Not exposed

Movie updates use `findByIdAndUpdate` with `runValidators: true`, so schema rules still apply to edits.

4.7 Authentication and Authorization

- Registration checks: name of at least 2 characters, valid email format, phone of exactly 10 digits if given, password of at least 8 characters, and a duplicate-email check that returns 409.
- Login returns the same message ("Incorrect email or password") for an unknown email or a wrong password.
- Every booking, payment and personal-listing route uses `requireAuth`, and queries filter by `req.user.id`, so users can only read their own bookings.
- Movie write routes use both `requireAuth` and `requireAdmin`.

Code snippet: authentication and role-check middleware (middleware/auth.js).

```
function requireAuth(req, res, next) {
  const header = req.headers.authorization || '';
  const token = req.cookies.cineworld_token ||
    (header.startsWith('Bearer ') ? header.slice(7) : null);
  if (!token) {
    return res.status(401).json({ message: 'Login required' });
  }
  try {
    req.user = jwt.verify(token, process.env.JWT_SECRET);
    next();
  } catch (err) {
    res.status(401).json({ message: 'Invalid token' });
  }
}

function requireAdmin(req, res, next) {
  if (req.user.role !== 'admin') {
    return res.status(403).json({ message: 'Admin only' });
  }
  next();
}
```

4.8 Booking and Seat Management

- **Validation:** seats are upper-cased, duplicates are rejected, and each seat must match the pattern `[A-E][1-8]`, which gives a 40-seat layout per show.
- **Holding seats:** reservations are inserted as `HELD` immediately when a booking is created.
- **Duplicate prevention:** the unique index on show + seat makes MongoDB reject a second reservation for the same seat, even if two requests arrive together. The code then cleans up and returns 409.
- **Seat map endpoint:** `GET /api/movies/shows/:id/seats` returns only seats whose status is `BOOKED`.

- **Status management:** `PENDING_PAYMENT` → `CONFIRMED` (after verified payment) or `CANCELLED` (manual cancel of an unpaid booking).

Code snippet: holding seats and handling a duplicate-seat conflict (routes/bookings.js).

```
const seatPattern = /^[A-E][1-8]$/;
if (!seats.every((s) => seatPattern.test(s))) {
  return res.status(400).json({ message: 'Invalid seat' });
}

try {
  await SeatReservation.insertMany(
    seats.map((seat) => ({
      show: show._id, seat, booking: booking._id, status: 'HELD'
    }))
  );
} catch (error) {
  // unique index rejected a seat: undo and report
  await SeatReservation.deleteMany({ booking: booking._id });
  await Booking.findByIdAndDelete(booking._id);
  return res.status(409).json({ message: 'A seat was just taken' });
}
```

4.9 Razorpay Test Payment Integration

- The gateway client is created only when test keys exist in the environment. If keys are missing or still contain placeholder text, `create-order` returns 503.
- The order amount is the booking total multiplied by 100 (paise) and currency is INR.
- Verification uses HMAC-SHA256 and `timingSafeEqual`. A failed check marks the payment `FAILED`.
- The integration uses Razorpay test keys only. Key values are kept in `.env` and are not included in this report.

Code snippet: server-side payment signature verification (routes/payments.js).

```
const expected = crypto
  .createHmac('sha256', process.env.RAZORPAY_KEY_SECRET)
  .update(`${razorpay_order_id}|${razorpay_payment_id}`)
  .digest('hex');

const valid = expected.length === razorpay_signature.length &&
  crypto.timingSafeEqual(
    Buffer.from(expected), Buffer.from(razorpay_signature));

if (!valid) {
  payment.status = 'FAILED';
  await payment.save();
  return res.status(400).json({ message: 'Verification failed' });
}

payment.status = 'PAID';
booking.status = 'CONFIRMED';
await Promise.all([
  payment.save(),
  booking.save(),
  SeatReservation.updateMany(
    { booking: booking._id }, { status: 'BOOKED' })
]);
```

4.10 Ticket Resale Management

- Only a seat from a `CONFIRMED` booking owned by the user can be listed.
- The listing records movie, date, time, seat and asking price.
- On purchase, the listing is claimed with a conditional update, the seller's booking loses that seat (and becomes `CANCELLED` if no seats remain), the buyer gets a new confirmed booking, and the unique seat reservation is re-pointed to the new booking.
- The buyer-side booking total is set to the asking price, but **no payment is collected**. This is a demo ownership transfer.
- The transfer is made of several separate database updates with a best-effort rollback, not a database transaction.

4.11 Environment Configuration

Configuration is read from a `.env` file, with a safe template in `.env.example`.

Table 4.5: Environment variables

Variable	Purpose
PORT	Server port (5000)
MONGODB_URI	MongoDB connection string for the cineworld database
JWT_SECRET	Secret for signing tokens
RAZORPAY_KEY_ID	Razorpay test key id
RAZORPAY_KEY_SECRET	Razorpay test key secret

Variable	Purpose
RAZORPAY_WEBHOOK_SECRET	Optional, for the webhook route

The `.gitignore` excludes `node_modules`, `.env` and the log file. Actual values are not shown in this report.

4.12 MongoDB Database Initialization

The database is created when the server first connects and data is written. The seed script `npm run seed` connects to MongoDB and:

1. Upserts 12 movies: 4 `now-showing` and 8 `coming-soon`, each with a price and image name.
2. Creates shows for the next 14 days for every now-showing movie at four times: 10:00 AM, 1:00 PM, 4:00 PM and 7:00 PM on "Screen 1" with capacity 40.
3. Prints a confirmation and disconnects.

Because it uses upserts, running the script again does not create duplicate movies or shows. Typical commands (PowerShell, from the project folder):

```
npm install
npm run seed
npm run dev
```

5. RESULTS AND EXECUTION OUTPUTS

The CineWorld movie ticket booking and resale application was implemented and tested using Visual Studio Code, Node.js, Express.js, MongoDB, a web browser, and Postman. The following results demonstrate the execution of the application, database connectivity, API operations, user authentication, movie booking, payment processing, and ticket resale functionality.

5.1 Backend Server Startup

The backend server was executed from PowerShell. The application successfully connected to MongoDB and started on port 5000.

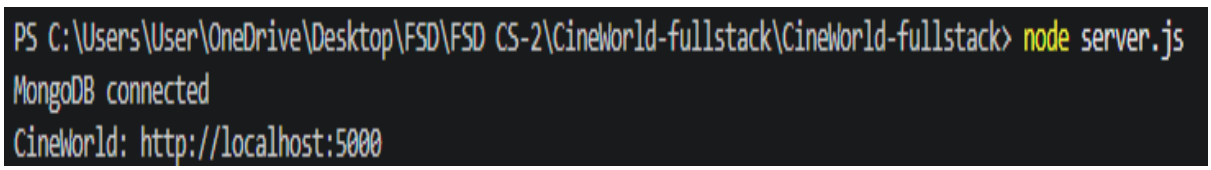


Figure 5.1: Backend server execution showing successful startup and MongoDB connection.

5.2 Health-Check API

The GET `/api/health` endpoint was tested to verify that the CineWorld backend was running correctly. The API returned a successful response containing the application status.

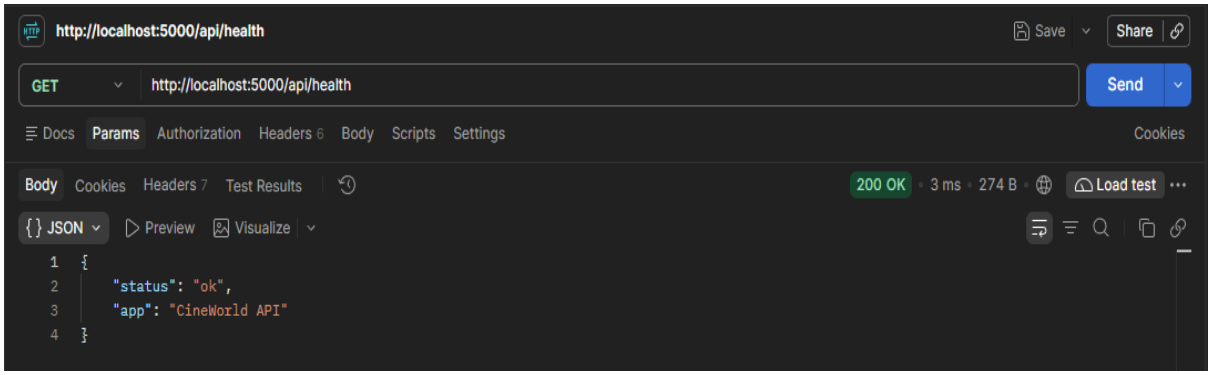
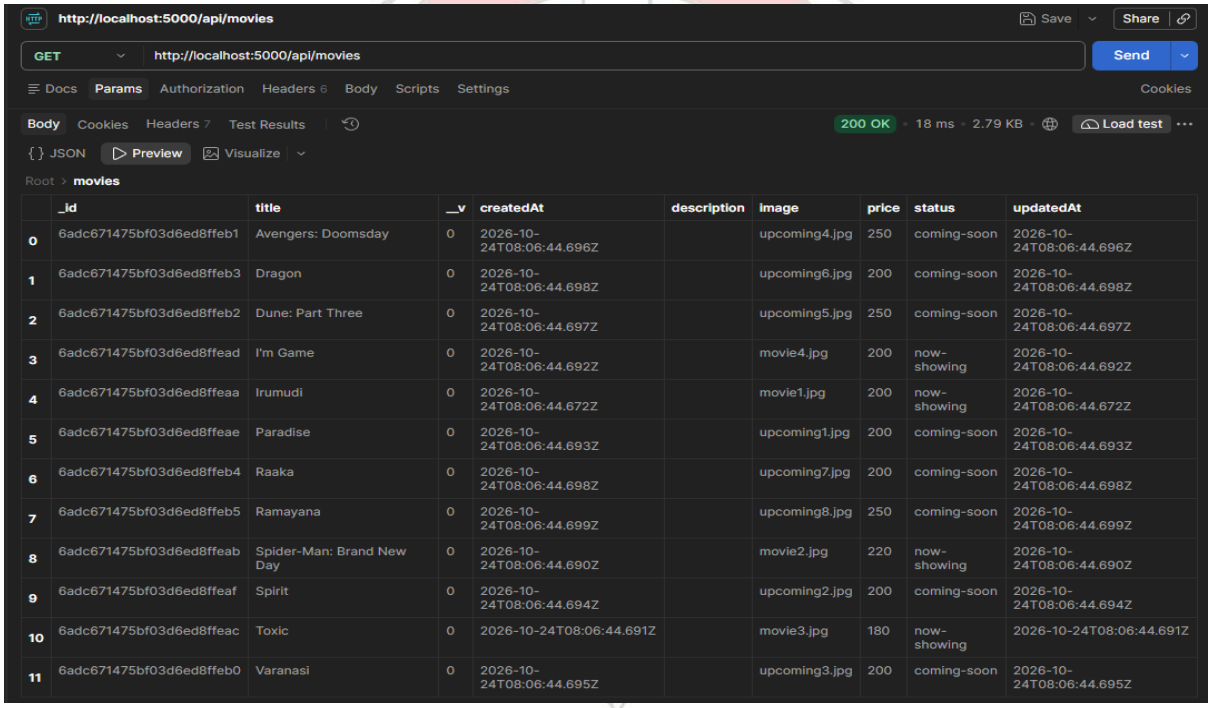


Figure 5.2: Health-check API response confirming that the CineWorld backend is operational.

5.3 Movie Retrieval Using GET

The GET `/api/movies` endpoint was tested using Postman. It retrieved movie details stored in MongoDB and returned them in JSON format. The endpoint also supports filtering movies by status.



	_id	title	__v	createdAt	description	image	price	status	updatedAt
0	6adc671475bf03d6ed8ffeb1	Avengers: Doomsday	0	2026-10-24T08:06:44.696Z		upcoming4.jpg	250	coming-soon	2026-10-24T08:06:44.696Z
1	6adc671475bf03d6ed8ffeb3	Dragon	0	2026-10-24T08:06:44.698Z		upcoming6.jpg	200	coming-soon	2026-10-24T08:06:44.698Z
2	6adc671475bf03d6ed8ffeb2	Dune: Part Three	0	2026-10-24T08:06:44.697Z		upcoming5.jpg	250	coming-soon	2026-10-24T08:06:44.697Z
3	6adc671475bf03d6ed8ffead	I'm Game	0	2026-10-24T08:06:44.692Z		movie4.jpg	200	now-showing	2026-10-24T08:06:44.692Z
4	6adc671475bf03d6ed8ffeaa	Irumudi	0	2026-10-24T08:06:44.672Z		movie1.jpg	200	now-showing	2026-10-24T08:06:44.672Z
5	6adc671475bf03d6ed8ffeae	Paradise	0	2026-10-24T08:06:44.693Z		upcoming1.jpg	200	coming-soon	2026-10-24T08:06:44.693Z
6	6adc671475bf03d6ed8ffeb4	Raaka	0	2026-10-24T08:06:44.698Z		upcoming7.jpg	200	coming-soon	2026-10-24T08:06:44.698Z
7	6adc671475bf03d6ed8ffeb5	Ramayana	0	2026-10-24T08:06:44.699Z		upcoming8.jpg	250	coming-soon	2026-10-24T08:06:44.699Z
8	6adc671475bf03d6ed8ffeab	Spider-Man: Brand New Day	0	2026-10-24T08:06:44.690Z		movie2.jpg	220	now-showing	2026-10-24T08:06:44.690Z
9	6adc671475bf03d6ed8ffeaf	Spirit	0	2026-10-24T08:06:44.694Z		upcoming2.jpg	200	coming-soon	2026-10-24T08:06:44.694Z
10	6adc671475bf03d6ed8ffeac	Toxic	0	2026-10-24T08:06:44.691Z		movie3.jpg	180	now-showing	2026-10-24T08:06:44.691Z
11	6adc671475bf03d6ed8ffeb0	Varanasi	0	2026-10-24T08:06:44.695Z		upcoming3.jpg	200	coming-soon	2026-10-24T08:06:44.695Z

Figure 5.3: GET operation retrieving movie records from the CineWorld database.

5.4 Showtime Retrieval Using GET

The GET `/api/movies/shows` endpoint retrieves available showtime information. The application also provides GET `/api/movies/shows/:id/seats` to retrieve booked-seat information for a particular show.

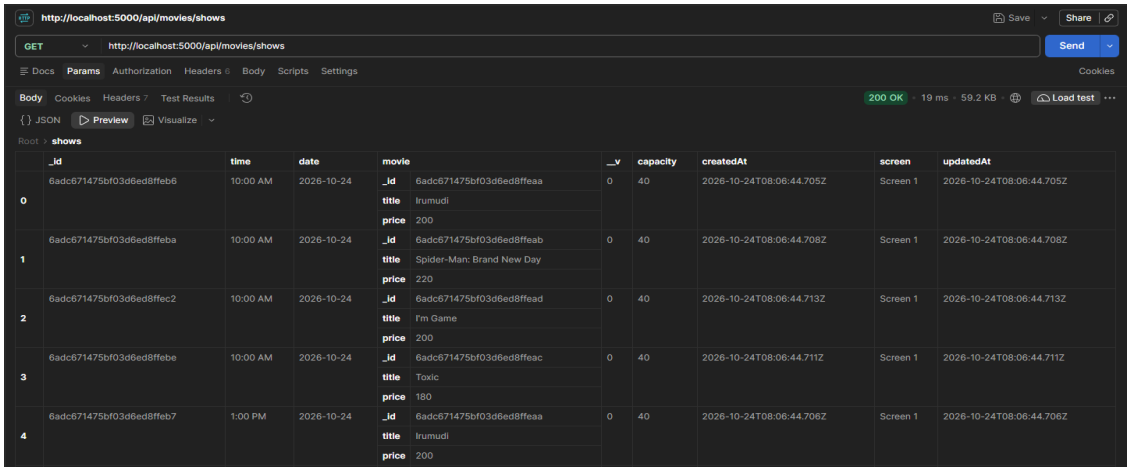


Figure 5.4: GET operation retrieving movie showtimes.

5.5 MongoDB Database and Collections

MongoDB was used to store application data. MongoDB Compass was used to inspect the `cineworld` database and verify stored movie, user, showtime, booking, payment, seat reservation, and resale records, where present.

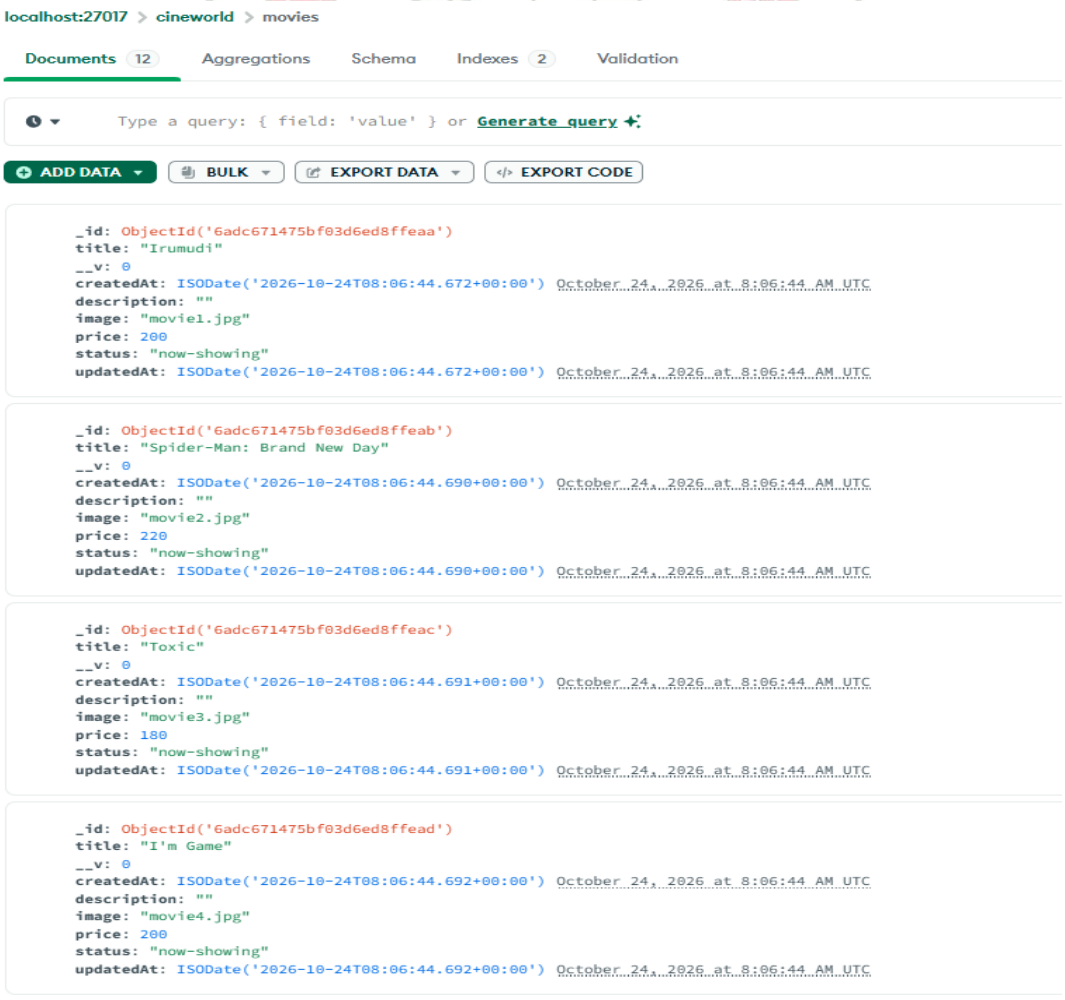


Figure 5.5: MongoDB Compass displaying the CineWorld database and collections.

5.6 User Registration and Login

The authentication module supports user registration and login. Passwords are stored as bcrypt hashes, and authenticated sessions use JWT-based authentication through an HTTP-only cookie.

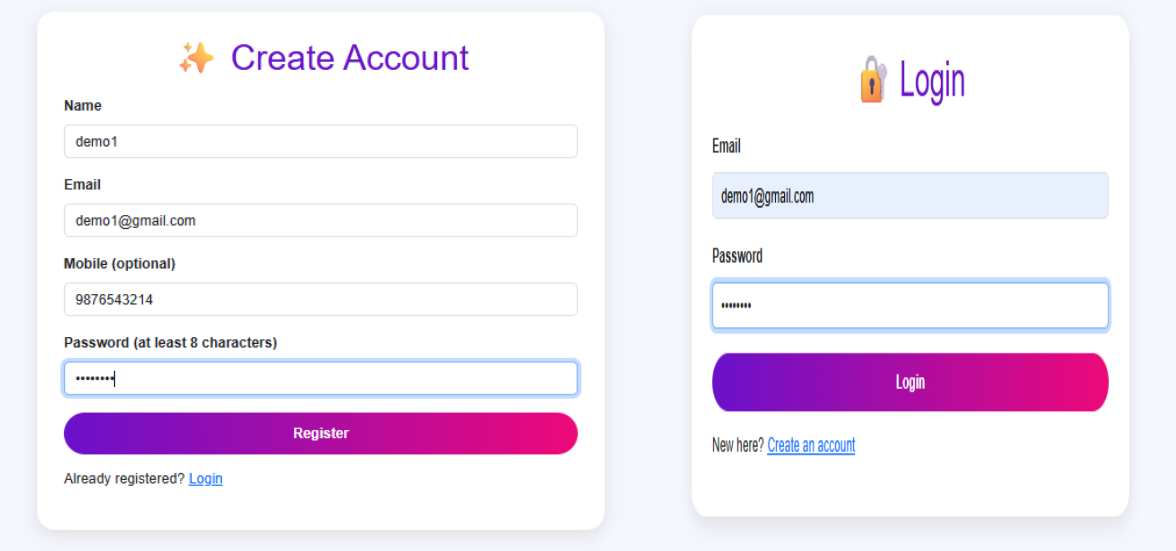


Figure 5.6: CineWorld user registration and login functionality.

5.7 Account Login Using POST

The POST `/api/auth/login` endpoint allows an authorized administrator to add a new movie to the database. The request contains movie details in JSON format. On successful creation, the API returns the created movie record with HTTP status `201 Created`.

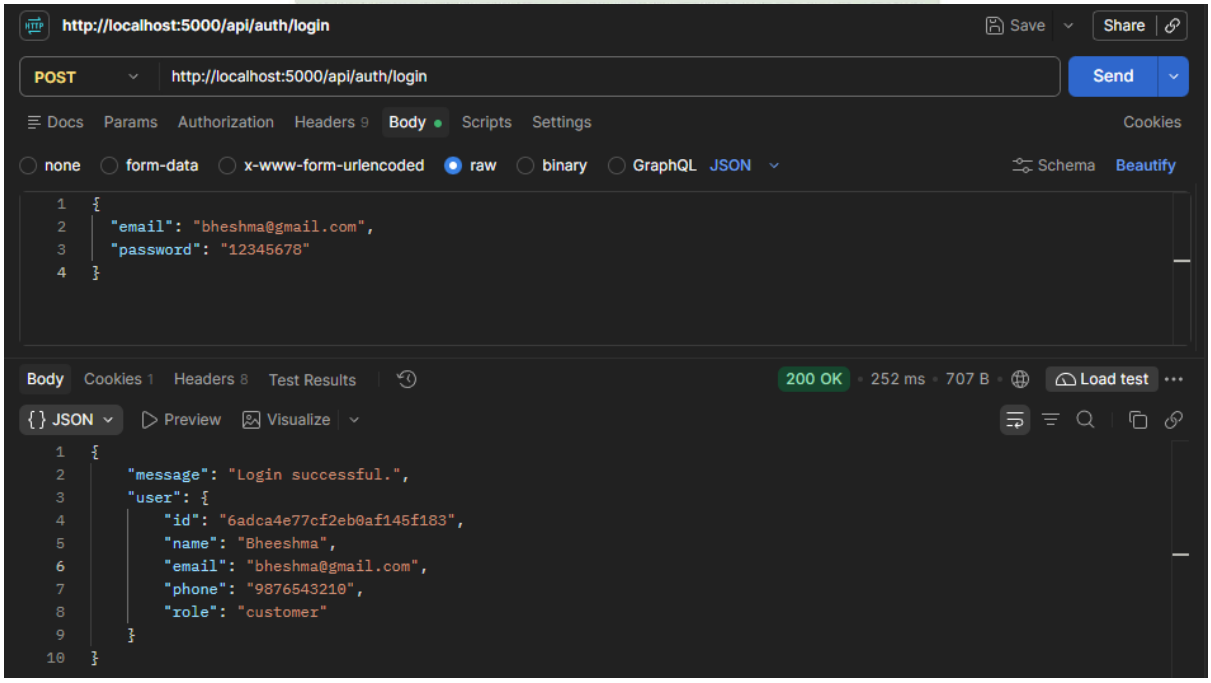


Figure 5.7: POST operation for account login authorization.

5.8 Movie Booking and Seat Selection

The booking module allows users to select a movie, showtime, and available seats. The total ticket price is calculated according to the selected seats. The backend creates booking records and reserves seats to help prevent duplicate seat bookings.

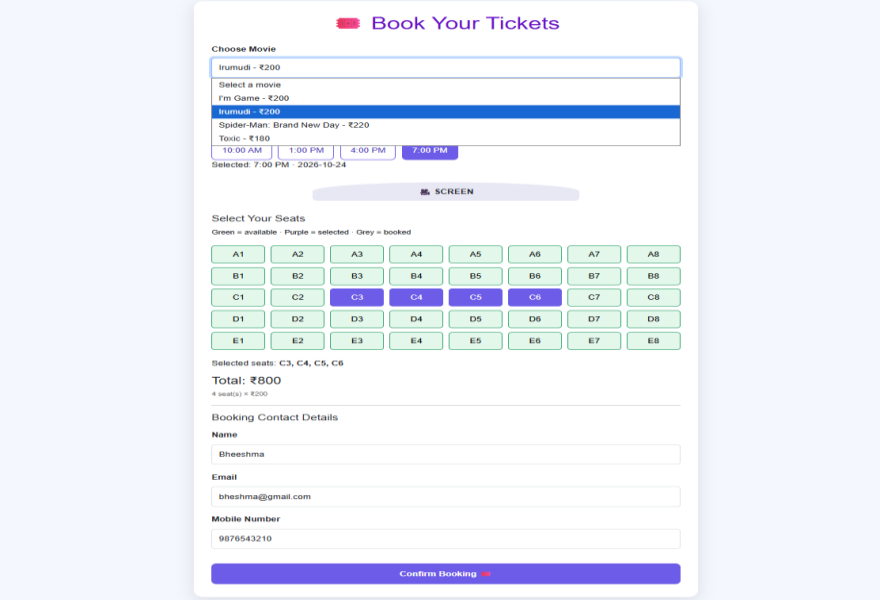


Figure 5.8: Movie booking interface with showtime and seat selection.

5.9 Booking Confirmation and MongoDB Records

After a successful test-mode payment, the booking was confirmed and displayed on the user's account page. Booking and seat reservation records can also be inspected in MongoDB Compass.

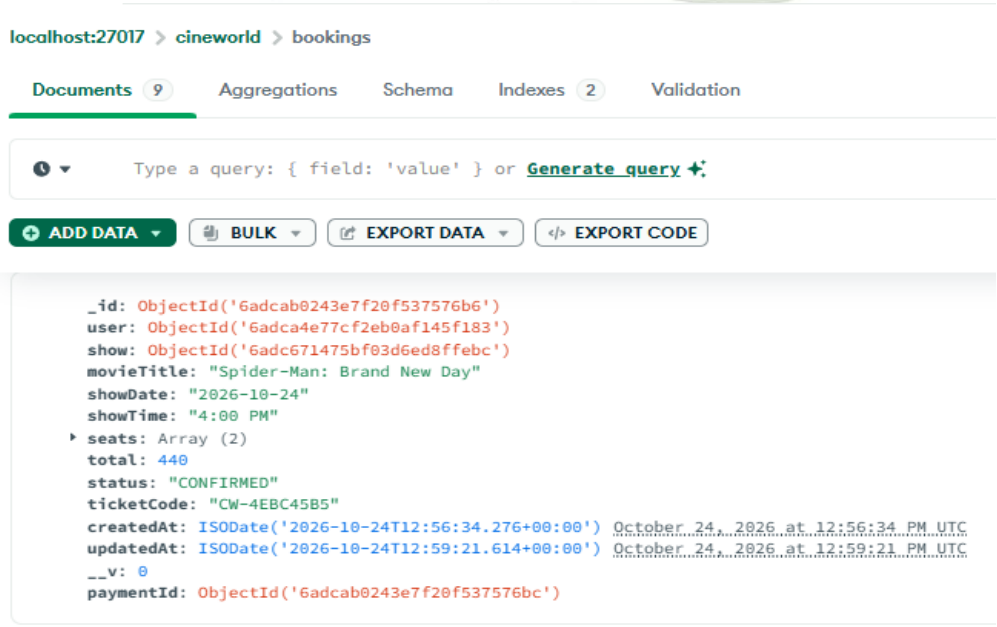
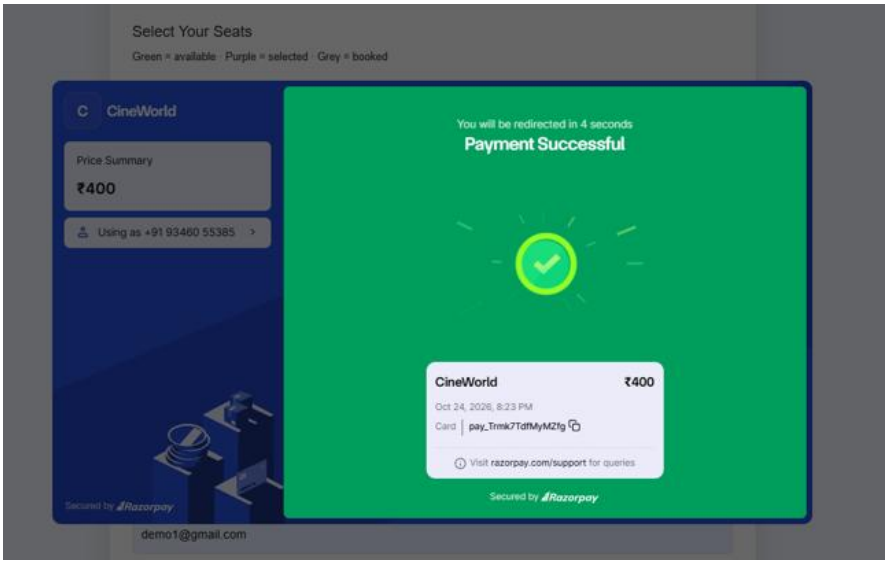


Figure 5.9: Confirmed booking and associated database records.

5.10 Razorpay Test-Payment Integration

The application integrates Razorpay for test-mode payment processing. The backend provides endpoints for creating a payment order and verifying the payment signature before confirming a booking.



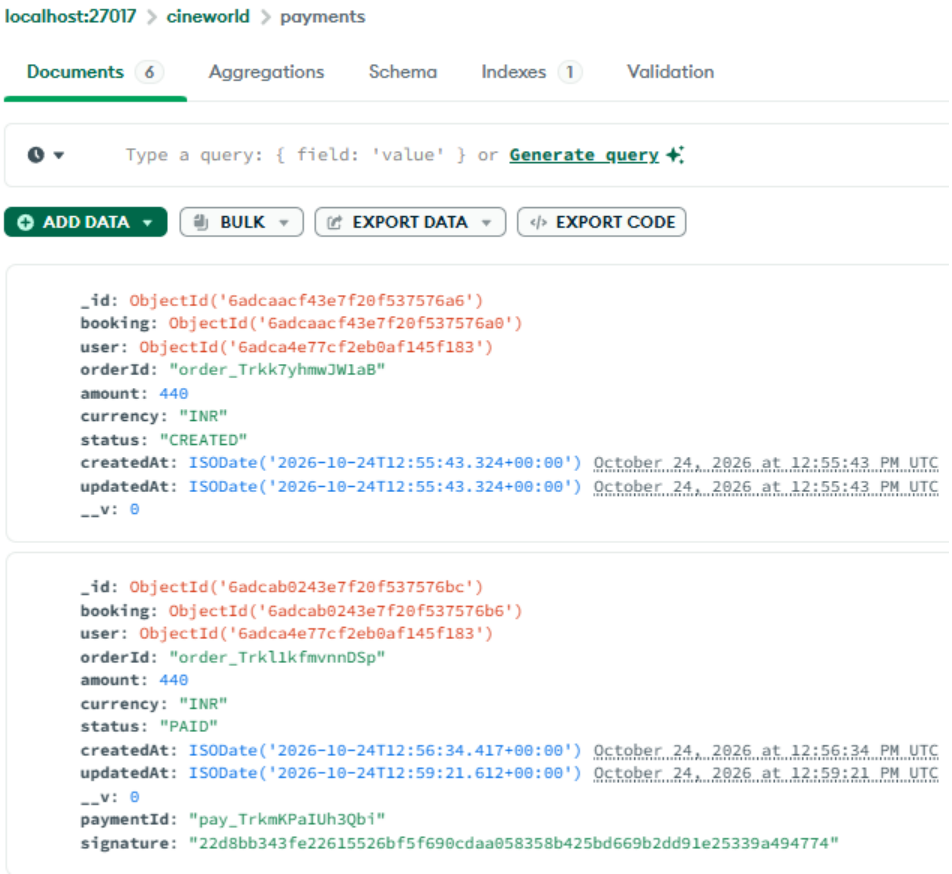


Figure 5.10: Razorpay test-mode payment processing and verification.

5.11 Movie Update Using PUT

The PUT `/api/movies/:id` endpoint was tested in Postman with administrator authorization. The operation updated the selected movie's details, and the API returned the updated movie record.

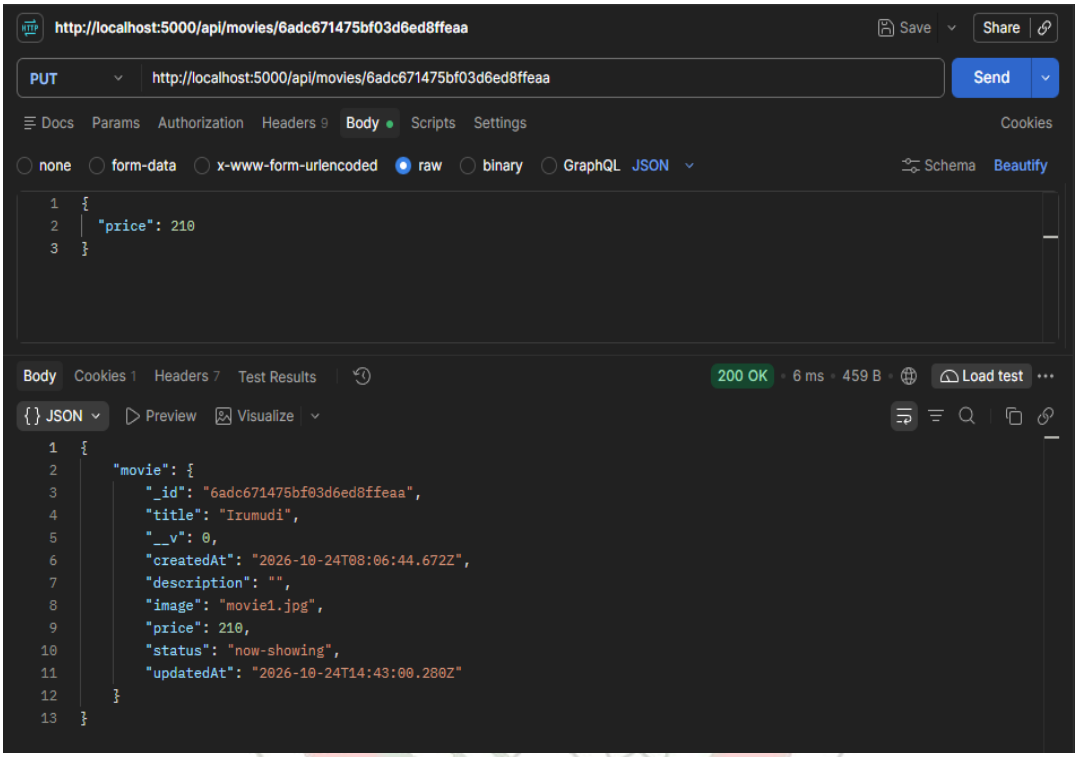


Figure 5.11: PUT operation for updating movie details.

5.12 Movie Deletion Using DELETE

The DELETE `/api/movies/:id` endpoint was tested in Postman with administrator authorization. The operation deleted the selected movie record and returned a success message.

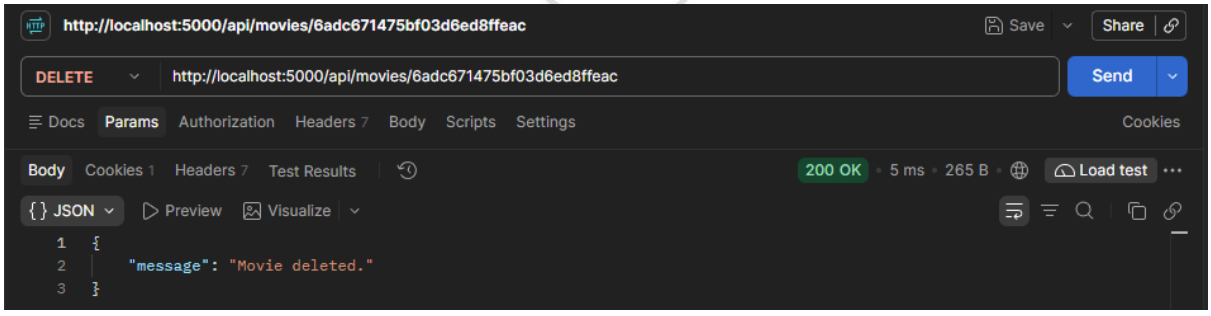


Figure 5.12: DELETE operation for removing a movie record.

5.13 Ticket Resale System

The CineWorld resale module supports listing eligible tickets, viewing available resale listings, purchasing a listed ticket through the ownership-transfer workflow, and withdrawing a listing. The system updates ticket ownership and resale status.

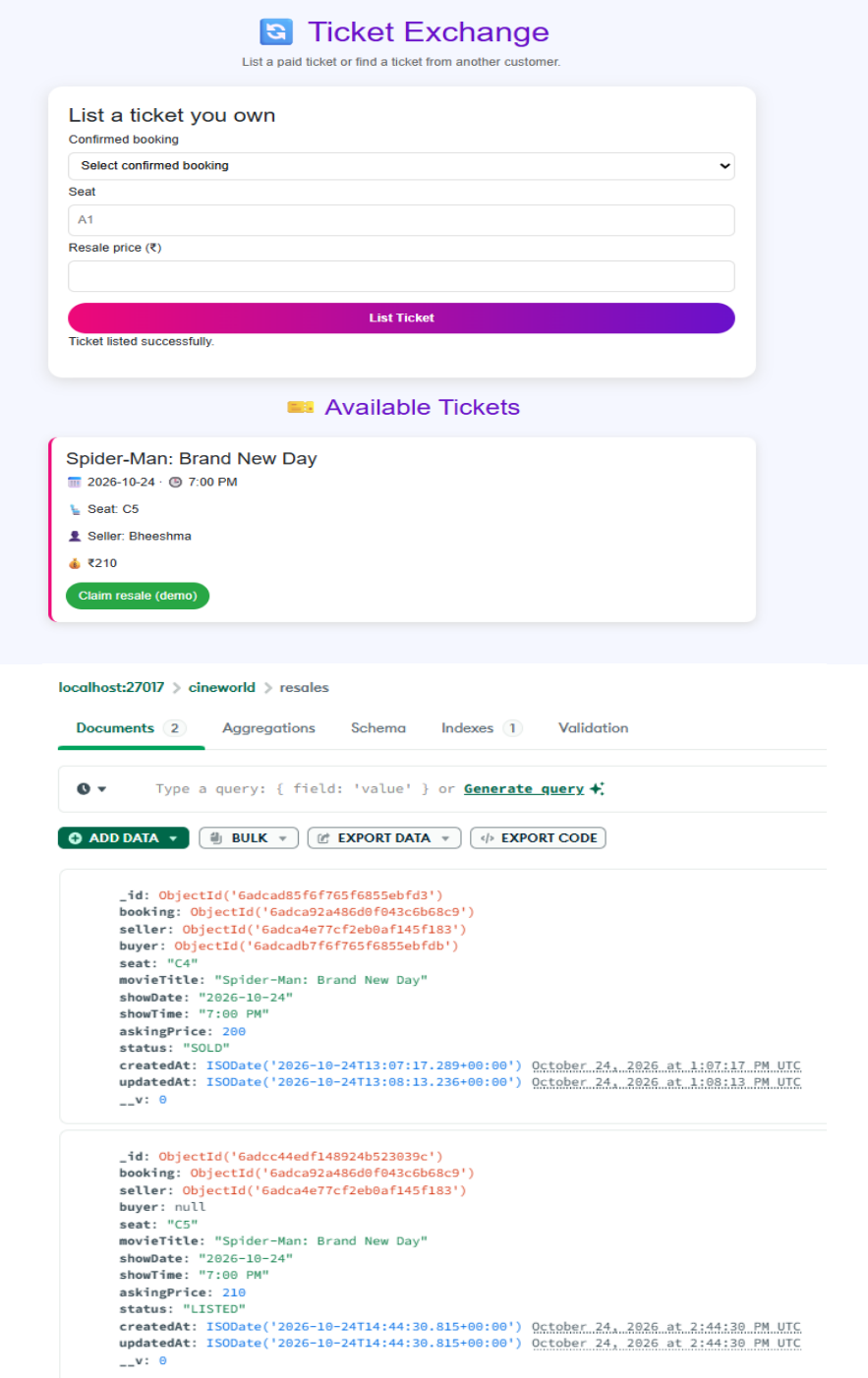


Figure 5.13: Ticket resale listing and ownership-transfer functionality.

Note: The implemented resale purchase workflow transfers ownership but does not collect a separate resale payment.

5.14 Final VS Code Project Structure

The project structure was inspected in Visual Studio Code. The application contains separate folders for authentication middleware, MongoDB models, backend API routes, frontend pages and assets, and database seed scripts.

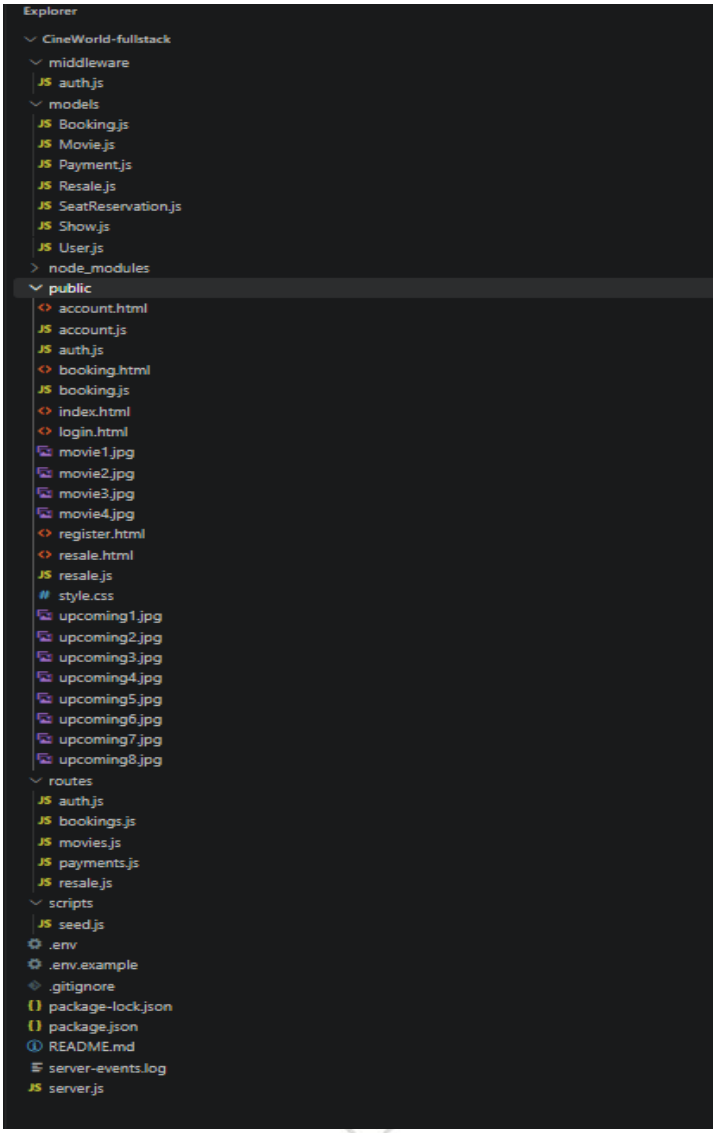


Figure 5.14: Final CineWorld project structure in Visual Studio Code.

5.15 Result Summary

The CineWorld application was executed locally and connected successfully to MongoDB. The demonstrated features include health-check verification, movie and showtime retrieval, user registration and login, movie creation, booking and seat selection, test-mode payment processing, administrator-protected movie updates and deletion, and ticket resale functionality. MongoDB stores the application's relevant data, while the Express.js backend provides API endpoints connecting the frontend with the database.The testing results demonstrate the integration of frontend components, backend REST APIs, authentication, database operations, booking management, payment verification, and resale functionality.

6. CONCLUSION

6.1 Summary of Findings

CineWorld is a working Node.js, Express.js and MongoDB backend for booking movie tickets and listing tickets for resale. Its code covers user accounts, protected routes, movie and show data, booking with seat validation, test-mode payment verification and a demo resale workflow. The results in Section 5 show which of these were executed and checked in the test environment.

6.2 Technical Findings

- A unique compound index on show + seat enforces one reservation per seat at the database level, so it holds even under simultaneous requests.
- Seats are held first and confirmed only after server-side signature verification, so the browser alone cannot confirm a booking.
- JWTs in HttpOnly cookies keep the token away from page scripts, and bcrypt hashing protects stored passwords.
- Role-based middleware separates admin movie management from normal user actions.
- The resale purchase uses an atomic conditional update to claim a listing, which stops two buyers from buying the same one.
- A custom `server-started` event with asynchronous file appending is implemented in `server.js`.

6.3 Learning Outcomes

- Structuring a Node.js and Express project into routes, middleware and models.
- Designing related Mongoose schemas with validation and indexes.
- Implementing JWT and cookie-based authentication with role checks.
- Integrating a payment gateway in test mode and verifying signatures safely.
- Handling race conditions with unique indexes and conditional updates.
- Inspecting data with MongoDB Compass and testing APIs from PowerShell.

6.4 Current Implementation Limitations

Table 6.1: Current implementation limitations

Limitation	Detail
Test mode only	Razorpay is used with test keys; this is not a production payment system
No resale payment	Resale purchase records an ownership transfer only; no money is collected
No transactions	Resale transfer uses several updates with best-effort rollback, not a MongoDB transaction
No hold expiry	Seats held for an unpaid booking stay held until the booking is cancelled

Limitation	Detail
Held seats hidden	The seat-map endpoint returns only BOOKED seats, so held seats are not shown as taken
No admin creation route	Registration creates only customers; the admin role must be set directly in the database
Webhook is minimal	The webhook route only validates a signature and does not update bookings
No automated tests	No test suite is included in the project files
Fixed seat layout	Seats are limited to rows A–E and numbers 1–8

6.5 Future Scope

These are proposed improvements and are **not** part of the current implementation:

1. Add a dedicated resale payment flow and use MongoDB transactions for ownership transfer.
2. Add automatic expiry of abandoned seat holds.
3. Make the webhook update payment and booking status.
4. Show held seats in the seat map.
5. Add an admin-management route or admin dashboard.
6. Add automated tests, rate limiting and request validation libraries.
7. Add email or SMS ticket delivery and QR-code tickets.
8. Deploy with a hosted database and production payment keys after security review.

6.6 Final Conclusion

CineWorld demonstrates the main parts of a full stack booking application: a REST backend, a document database with enforced seat uniqueness, authenticated access, test-mode payment verification and a controlled resale demo. It is a solid educational base. The limitations listed above, especially the demo-only resale transfer and test-mode payments, would need to be solved before any real-world use.